# COMP30027 Report:
# Traffic Sign Classification

## 1. Introduction

The purpose of this report is to implement various models in a traffic sign classification task to label traffic signs into one of 43 classes. The process began with an exploratory analysis of the data, to help understand the data better. This was followed by additional feature engineering and selection. These features were utilized in the implementation of the models, namely Support Vector Machines (SVM), k-Nearest Neighbors (kNN) and Convolutional Neural Network (CNN). These models were each followed by an evaluation.

## 2. Methodology

### 2.1 Exploratory Data Analysis

An initial exploration of the data was conducted. The instances of each class were plotted to visualize any underlying distribution within the training data. The plot also emphasised any imbalances of class representation.

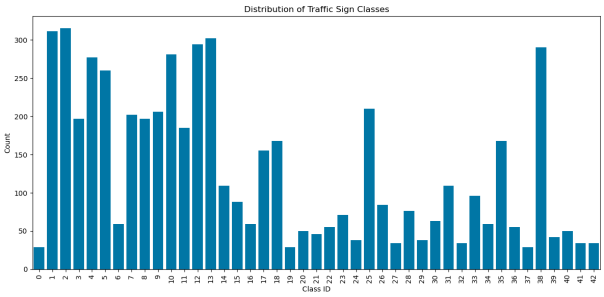



**Fig. 1-** Distribution of traffic sign cases

| ClassId | Count |
|---|---|
| Top 3 | |
| 2 | 315 |
| 1 | 311 |
| 13 | 302 |
| Bottom 3 | |
| 19 | 29 |
| 0 | 29 |
| 37 | 29 |

**Table 1-** Most and least frequent classes

Signs of class 2, 1, and 13 were most represented in the training data, with 315, 311, and 302 instances respectively. In comparison, class 19, 0, and 37, were significantly underrepresented, each only having 29 instances in the training dataset.

### 2.2 Feature Engineering

In addition to the training data, supplementary datasets were provided, which specified features that could enhance the performance of the machine learning models. To see if any of these existing features would be useful, mutual information (MI) scores of the features against the class label were determined. MI quantifies the amount of information one random variable contains about another random variable (Brownlee, 2019). In the dataset 'additional_features.csv', we were given the features; 'edge_density', 'mean_r', 'mean_g', and 'mean_b'.

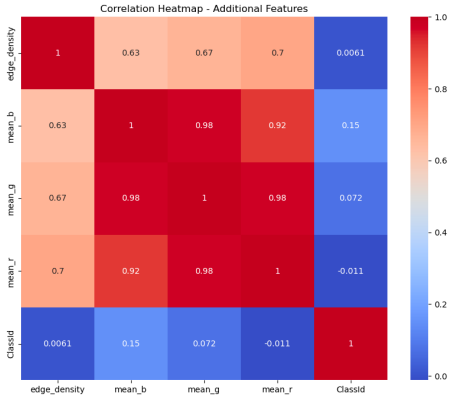



**Fig. 2-** Heatmap of mutual information scores

We found that mean_b is most strongly predictive. To further enhance model performance, additional features were engineered including colour ratios, shape energy, and edge-to-colour ratios.

Colour ratios were calculated by:

$$colour_ratio = \frac{mean_c}{mean_r + mean_g + mean_b} \; with \; c \in \{r, g, b\}$$

This aims to measure the intensity of the colour. Edge-to-colour measures how textured an image is relative to its overall brightness or colour, and is defined as:

$$edge_to_colour = \frac{edge_density}{mean_c} \; with \; c \in \{r, g, b\}$$

Shape energy is the Euclidean norm of the vector of hog_pca values, suggesting how strong or complex the overall shape patterns are.

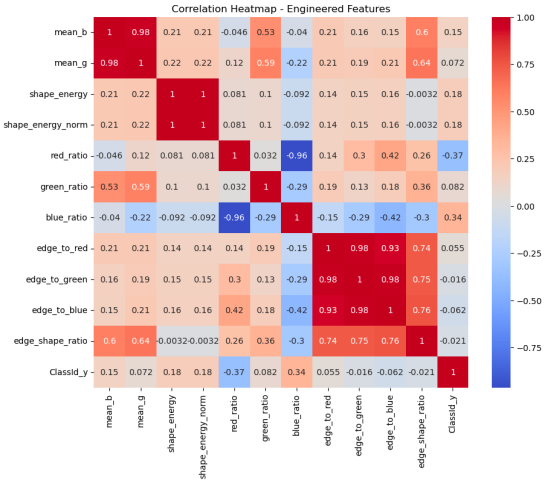



**Fig. 3-** Heatmap of mutual information scores of the engineered features

We can see that the most predictive features for ClassId are red_ratio, blue_ratio, and shape_energy. This is reasonable as most traffic signs contain red and/or blue hues. Shape energy is quite strongly predictive as well. This is also reasonable since the shape of a sign is often indicative of its class.

The features which had a MI score below 0.02 were filtered, and the remainder were chosen for implementation in the models.

## 2.3 Machine Learning Models

### 2.3.1 SVM

Support Vector Machine (SVM) is a machine learning model that plots instances onto a plane and these instances based on a hyperplane that can separate the instances from each other (Ehinger, 2025). Given that this is a multi-class context, a OnevOne (OvO) classification model was implemented. Parameters were hyper tuned to optimise its performance.

### 2.3.2 k-NN

K-Nearest Neighbours (k-NN) is a model that 'assumes all instances correspond to points in the n-dimensional space' where it classifies an instance based on its k nearest neighbours (Mitchell, 1997). Parameters such as split ratio, and choice of k value were hyper tuned to optimize its performance. Euclidean distance was chosen for the distance metric.

### 2.3.3 CNN

The convolutional neural network model is a 'specialized type of deep learning algorithm designed to process data with a grid-like topology, such as images' (IBM, n.d). Since this model feeds the actual image data itself, it did not necessitate any features to be added. It implicitly learns its own features. A dropout of 0.5 was chosen.

# 3. Results

## 3.1 SVM

### 3.1.1 SVM with original hog_pca

| | precision | recall | F1 | support |
|---|---|---|---|---|
| accuracy | | | 0.87 | 1098 |
| macro avg | 0.88 | 0.86 | 0.87 | 1098 |
| weighted avg | 0.88 | 0.87 | 0.87 | 1098 |

**Table 2-** classification report for SVM with provided hog_pca values

This model worked well on the validation set but performed significantly worse on the test set. It yielded an accuracy of 0.87, and precision of 0.88. On the test set, the model performed with an accuracy of 0.30. This discrepancy prompted further investigation into the feature extraction process, specifically whether extracting hog_pca values manually could improve model performance.

### 3.1.1 SVM with created hog_pca values

We then implemented a model using manually extracted hog_pca values.

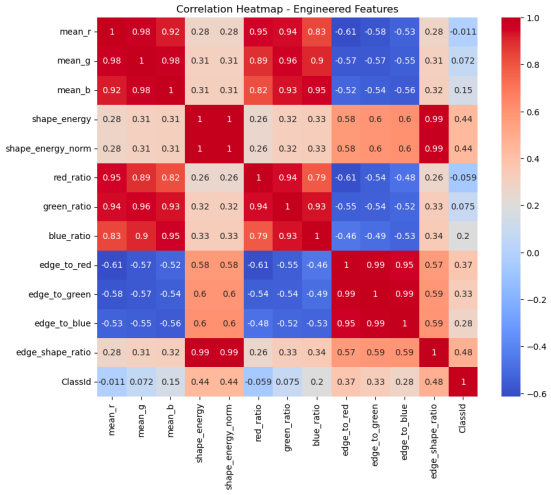



**Fig. 4-** Heatmap of mutual information scores of the engineered features with the extracted hog_pca values

The new hog_pca values were then used to engineer the features from before.

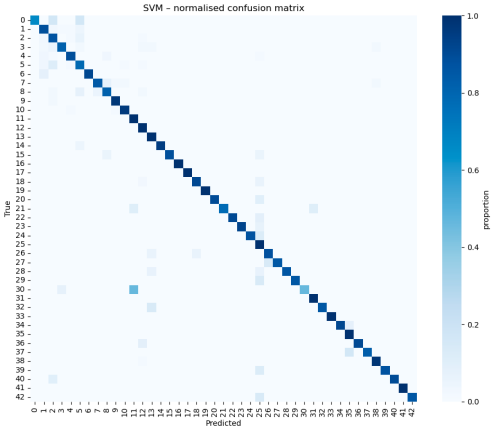



**Fig. 5-** Confusion matrix of the performance of the SVM model

These features were again added to our final SVM model. Evidenced by the Fig.5, our model has low bias but high variance, since classification errors are random. However, a prominent proportion of signs of class 30 are being mistaken for 11.

| | **precision** | **recall** | **F1** | **support** |
|---|---|---|---|---|
| accuracy | | | 0.917 | 1098 |
| macro avg. | 0.958 | 0.899 | 0.923 | 1098 |
| weighted avg | 0.923 | 0.917 | 0.916 | 1098 |

**Table 3-** Classification report for SVM

This model performs better overall than the initial SVM. In this case, the performance on the validation set reflected the performance on the test set.

| Class | Value | Support |
|---|---|---|
| 30 | 0.462 | 13 |
| 0 | 0.667 | 6 |
| 21 | 0.778 | 9 |

**Table 4-** Bottom 3 recall values for SVM

### 3.2 k-NN

| | **test_frac** | **best_k** | **cv_acc** | **test_acc** |
|---|---|---|---|---|
| 0 | 0.10 | 1 | 0.863335 | 0.894353 |
| 1 | 0.15 | 1 | 0.850346 | 0.887136 |
| 2 | 0.20 | 1 | 0.847153 | 0.877960 |
| 3 | 0.25 | 1 | 0.837946 | 0.876093 |
| 4 | 0.30 | 1 | 0.832597 | 0.865817 |

**Table 5-** Table with the hyperparameters that yielded the best kNN result

The best k value was 1, and the best split was 0.10, which performed the best in both cross-validation (CV) and overall validation set accuracy.

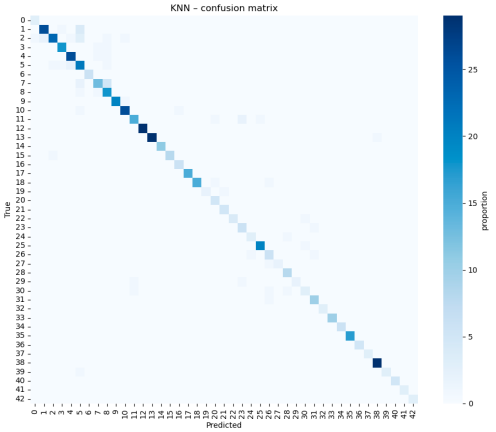



**Fig. 6-** Confusion matrix of the performance of the kNN model

Fig.6 outlines the low variance but high bias of the model. Its performance is best for class 38, 13, and 12.

| | **precision** | **recall** | **F1** | **support** |
|---|---|---|---|---|
| accuracy | | | 0.89 | 549 |
| macro avg. | 0.90 | 0.89 | 0.89 | 549 |
| weighted avg | 0.91 | 0.89 | 0.89 | 549 |

**Table 6-** Classification report for the kNN

The overall performance metrics of the model can be seen in the table above.

| Class | Value | Support |
|---|---|---|
| 29 | 0.5 | 4 |
| 30 | 0.5 | 6 |
| 2 | 0.72 | 32 |

**Table 7-** Bottom 3 recall values for kNN

### 3.3 CNN

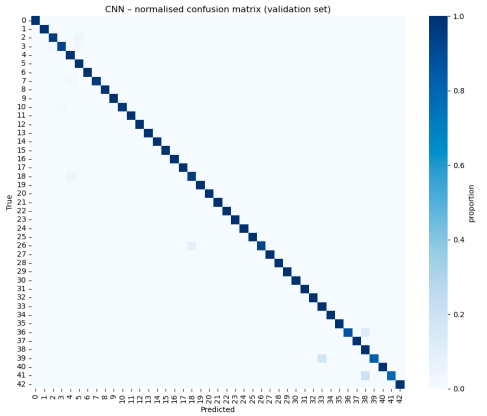


**Fig. 7-** Confusion matrix of the performance for CNN

The CNN model had minimal variance and low bias.

| | **precision** | **recall** | **F1** | **support** |
|---|---|---|---|---|
| acc | | | 0.987 | 824 |
| Macro avg | 0.988 | 0.984 | 0.985 | 824 |
| Weighted avg | 0.988 | 0.987 | 0.987 | 824 |

**Table 8-** Classification report for the CNN

This model had the best performance overall, among the three models implemented. It had a 0.987 accuracy on the validation set and had 0.9919 on the test set.

| Class | Value | Support |
|---|---|---|
| 21 | 0.714 | 7 |
| 1 | 0.957 | 47 |
| 18 | 0.96 | 25 |

**Table 9-** Bottom 3 recall values for CNN

# 4 Discussion and critical analysis

## 4.1 SVM

### 4.1.1 HOG consistency

We began the implementation of this model using the provided hog_pca embeddings. A radial basis function (RBF) SVM, tuned using 5 fold cross validation on the training split, achieved a 0.92 validation accuracy, but only 0.30 on the blind test set.

This large gap pointed to either overfitting to the training distribution or insufficient representational power in the original embeddings. This encouraged us to regenerate HOG descriptors from the raw images, and create our own hog_pca values. Using these new values, the validation and test performance aligned, indicating that the mismatch was feature related. In pursuit of consistency, these new hog_pca values were then used to engineer the features outlined in section (2.2). While the most predictive features remain, they have greater MI values, and are more strongly correlated to ClassId (Fig. 4). This consistency suggested improved generalizability and motivated the newly extracted hog_pca features to be used for the remainder of the investigation.

### 4.1.2 Model Implementation

Since the model was used for multiclass classification, the OvO was implemented. This is where the hyperplane separates every class against another class, repeating until all instances have been classified. A OnevRest (OvR) approach was also computed, but it yielded identical results. This suggests that the dataset was clean enough overall, so the choice of implementation did not have much impact. Furthermore, the feature engineering and filtering may have contributed to the model's ability to generalise well overall.

The scatter of the points suggests that some classes are harder to distinguish (Fig 5). These inconsistencies indicates that the decision boundaries may be highly sensitive to small variations in the input features. The model has a better precision than recall overall (Table 3) .



**Fig. 8-** Signs classified as class 11 (left) and class 30 (right)

For instance, class 30 has very poor recall, with a value of 0.462. Due to similar shape and relatively similar complexity in the sign, the model often mistakenly classifies it as class 11. Such resemblance can cause the model to overfit to specific patterns seen during training.

## 4.2 k-NN

### 4.2.1 Model Implementation

We continue to use the hog_pca values collected during the implementation of the SVM model. For the k-NN model, performance was highest when using a small value of k. When using a param grid to find the best hyperparameter values, k=1 yielded the best results (table. 1). This is likely due to the large number of possible classes in the dataset. When multiple neighbours become considered, the prediction may be distorted by class overlap, especially if the majority class dominates the neighbourhood. Moreover, k-NN models tend to struggle with high dimensional data due to "the curse of dimensionality". This is where distances between points become less meaningful and the model becomes more sensitive to noise (Mitchell, 1997). Even with the feature filtering, there are still a relatively high number of dimensions.

### 4.2.2 Model evaluation

Fig. 1 highlights the class imbalance, demonstrating that class 38 is well represented

in the training data, resulting in a high proportion of correct predictions (Fig. 6). Class 19 has relatively few training examples and correspondingly lower prediction accuracy. This issue could be addressed by oversampling underrepresented classes or applying class-balanced weighting.

This model also demonstrates low variance, but high bias, typical of k-NN models with small k values in high-dimensional contexts (insert citation). Its errors are systematic, consistently underfitting underrepresented classes while favouring majority ones. This is further supported by the model's precision and recall metrics.

Underrepresented classes like 19 had perfect precision, but recall values of 0.67. The model is cautious, only predicting these classes when very confident. This conservative prediction pattern stems from limited exposure to minority classes during training.

### 4.3 CNN

#### 4.3.1 CNN background

The CNN was the best performing model out of the 3 that were implemented, with validation and test accuracies of 0.987 and 0.9919 respectively. This is because it reads raw image pixels directly, learning its own hierarchical features through convolutional layers, unlike SVM and kNN, which rely on fixed feature extraction (IBM, n.d.). Consequently, the CNN model can capture subtle patterns and variations in traffic sign appearance more effectively.

#### 4.3.2 Model Implementation

The CNN was implemented using 3 convolutional blocks, giving a total of 6 convolutional layers. The shallow depth of the model keeps overfitting in check, but may under-fit if signs contain very fine detail.

We used a dropout rate of 0.5, which is a standard value for small-to-medium networks. This reduces overfitting, randomly deactivating half of the neurons during training. The model is prevented from relying too heavily on any specific pathway, forcing the model to learn robust and distributed representation. This redundancy makes the network more resilient to noise and reduces overfitting.

Training typically stabilized within 12 epochs, stopping with a patience of 3. This number was selected as it prevents the dense head from memorising noise in later epochs. 15% of data was reserved as a validation holdout to evaluate generalization during performance.

#### 4.3.3 Model Evaluation

While the model performs well as a whole, it is worth noting that the smallest metric is the macro avg. recall (Table 8).

From Fig. 7, we can see that a proportion of class 39 is being predicted as 33.



**Fig. 9-** Signs classified as class 11 (left) and class 30 (right)

These signs representing class 39 and 33 exhibit notable visual similarities, regarding colour, shape, and design complexity. Class 39 achieves a recall of 0.833. The model may be misled in these instances, due to pixel level cues, especially the ones near the bottom of class 33 (Fig. 9). The overall difference in resolution may also contribute to the classification challenges of this model.

## 5 Conclusions

Each model performed well and was able to generalize effectively to unseen data. The engineered features were helpful and contributed well to the overall performance. However, the inability to accurately classify certain classes was largely due to class imbalance, where underrepresented categories provided insufficient training signals. In the CNN model, additional factors such as the noise of the images and varying image resolutions also had impacted its performance. These limitations highlight the importance of both data quality and distribution in achieving reliable, equitable predictions.

## 6 References


Brownlee, J. (2019, April 29). *Information gain and mutual information*. Machine Learning Mastery. https://machinelearningmastery.com/information-gain-and-mutual-information/

Ehinger, Kris. (2025). *Week 5: Instance-based learning and SVM* [Lecture slides]. Canvas. https://canvas.lms.unimelb.edu.au/courses/211168/pages/week-5-instance-based-learning-and-svm?module_item_id=6327590

IBM. (n.d.). *What are convolutional neural networks?* IBM Think.

https://www.ibm.com/think/topics/convolutional-neural-networks

Mitchell, T. M. (1997). *Machine learning* (pp. 231–232). McGraw-Hill.